

PFC AI Design Agent

Future Expansion Plan & Architecture Principles

New chapters (semiconductors, protection, EMI) · Simulation integration
Extensibility rules that prevent redesign of existing work

The core principle: add without breaking

Version 1.0 · June 2026 · No source code changes

This document answers the question: "How do we plan for future chapters and simulation without forcing a redesign of what we have already built?" The answer is rooted in one architectural principle applied consistently at every layer of the system. If we follow this principle now, every future addition takes days not weeks — and never touches existing working code.

Part 1 — The One Principle That Prevents Redesign

All the specific rules in this document are applications of one underlying principle:

SEPARATION OF CONCERNS

Every layer of the system does exactly one job. Calculation modules compute. Report generators format. Frontend components display. Orchestrators combine. Documentation agents document. No layer reaches into another layer's job.

When every layer is independent, adding a new chapter means adding new files to each layer — it never means modifying existing files in that layer. The six layers of this system are:

Layer	Responsibility	File pattern	Adding a new chapter
1 — Calculation	Compute design values. Pure maths, no formatting, no UI.	step{N}_{topic}.py	Add step{N+1}_{topic}.py. Nothing else changes.
2 — Report generator	Format calculation results as PDF flowables. No calculations.	generate_step{N}.py	Add generate_step{N+1}.py. Nothing else changes.
3 — Orchestrator	Call each generator in sequence. Merge PDFs.	generate_combined_report.py	Add 3 lines: import, call, append. That is all.
4 — API endpoint	Expose calculation and report as HTTP endpoints.	main.py route + Pydantic model	Add one @app.post route. Existing routes unchanged.
5 — Frontend component	Display results. Collect designer input. Call API.	Step{N}Wizard.tsx	Add new component. App.tsx gets one new route entry.
6 — Documentation agent	Apply documentation standards to report generator files.	generate_step{N}.py (edit target)	Agent reads the new file, applies standards. No agent code changes.

INSIGHT The Three-Line Orchestrator Rule

Adding a new chapter to the combined report requires exactly three lines in generate_combined_report.py: one import, one calculation call, one PDF append. If adding a chapter requires more than that in the orchestrator, the separation of concerns has been violated somewhere.

Part 2 — Six Rules for Safe Extension

These rules must be followed when implementing any new step — now or in the future. Violating any one of them is the most common cause of a "small addition" turning into a "major rework".

✓ RULE Rule 1 — Define the Approved Result Contract First

Before writing a single line of calculation code for a new step, define exactly what the approved result object looks like. Write the TypeScript interface and the Python Pydantic model. Make them match. The result object is the contract between the frontend (which produces it at approval time) and the backend (which consumes it at report generation time). The Step 15 capacitor bug — where sections 15.7–15.16 were missing — happened because the approved result did not carry the `selected_cap` block. This is the most important rule.

✓ RULE Rule 2 — One Calculation Module Per Chapter, No Cross-Imports

`step7_magnetic_calc.py` does not import from `step15_capacitor.py`. `step16_control_design.py` does not import from `step7_magnetic_calc.py`. Each module is self-contained. If two modules need the same physics, put the shared function in a common utilities file (e.g. `pfc_utils.py`) that neither module owns. Cross-imports create hidden dependencies that make "adding a chapter" silently modify existing calculations.

✓ RULE Rule 3 — Report Generator Uses Only the Approved Result Dict

`generate_step{N}.py` receives one argument: the dict produced by the calculation module. It does not call any calculation function directly. It does not read from the database. It does not call other report generators. All data it needs must be in the approved result dict. If something is missing from the dict, fix the calculation module — do not add logic to the report generator.

✓ RULE Rule 4 — App.tsx is a Router, Not a Data Transformer

`App.tsx` maps step names to components and passes approved results through. It does not compute, transform, or summarise design data. If a future component needs a derived value (e.g. $\text{efficiency} \times \text{Pout}$), that value must be computed in the backend calculation module and included in the approved result — not computed in `App.tsx`.

✓ RULE Rule 5 — Simulation is a Review Step, Not a Parallel Track

Simulation fits naturally after an approval step as a "Review" sub-step, exactly like `ReviewMagnetics` follows the core selection. It uses the same `iframe+postMessage` architecture. It does not replace the calculation; it visualises it. Simulation results that the designer approves become figures in the report chapter.

✓ RULE Rule 6 — Never Modify a Passing Test When Adding a Feature

The calculation regression suite must continue to pass after every addition. If a new step requires changing an existing calculation module in any way, stop and redesign the new step. The existing module is correct by definition (it has passing tests). A new step should never need to change it.

Anti-Patterns That Force a Redesign

X ANTI-PATTERN Shared State Between Steps in the Backend

If step7 writes to a global dict and step15 reads from it, adding step17 requires understanding both step7 and step15 before writing a single line. Fix: pass all data explicitly through the request body. The state lives in the frontend (App.tsx) and is sent to each endpoint.

X ANTI-PATTERN Calculation Logic in the Report Generator

"I'll just compute the efficiency quickly here in generate_step16.py" — this is how a simple addition becomes a bug that takes a day to find. The report generator should contain zero physics. Zero maths. Only formatting.

X ANTI-PATTERN Frontend Components That Derive Design Values

"I'll compute L_full here in Step7Wizard since I have L0 and kbias" — this caused the L_full_load bug where enrichResult always returned L_target. Fix: the backend computes L_full and returns it. The frontend displays it.

X ANTI-PATTERN Hardcoded Step Count or Chapter Count Anywhere

if (step === 16) generateFullReport() — as soon as Chapter 7 is added, this condition is wrong. Fix: the report generation is triggered by the presence of approved results, not by a hardcoded step number.

Part 3 — Complete Chapter Roadmap

The following shows the full planned chapter roadmap — existing chapters (1–6, currently being implemented) and future chapters (7–9, planned). The colour scheme is consistent with the agreed chapter colours.

3.1 Existing Chapters (Currently Being Implemented)

Chapter 1 What must this design achieve?	Specifications → Design requirements and standards → Operating points matrix (9 points) → Compliance (IEC, SEMI F47)
Chapter 2 What architecture do we use?	Topology and Control Scheme Selection → Topology scoring and selection → CCM operating mode → Switching frequency rationale
Chapter 3 What inductor do we need?	PFC Inductor Sizing → Target inductance, currents, ripple → K(D) cancellation factor → Material, core, wire selection
Chapter 4 How does it behave?	PFC Inductor Performance Analysis → Waveforms: $H(t)$, $B(t)$, $P_{core}(t)$ → 3D model, window build → Loss vs V_{in} , thermal analysis
Chapter 5 How do we size the bank?	DC Bus Capacitor Selection → Hold-up and ripple sizing → Ripple current verification → Three-method lifetime analysis
Chapter 6 How do we close the loops?	Control Scheme → Control theory foundation (Section 6.0) → Current and voltage loop design → Stability scorecard

3.2 Future Chapters (Planned, Not Yet Implemented)

Chapter 7

What switches and diodes do we need?

Power Semiconductors

→ Bridge rectifier — I_{avg} , I_{rms} , V_R , dissipation → MOSFET — V_{DS} , $I_{D,pk}$, $R_{DS,on}$, gate charge, switching loss → Body diode / anti-parallel diode — reverse recovery, $I_{F,avg}$ → Heatsink sizing and junction temperature verification → Supplier data: datasheet parameters, SPICE model extraction

Chapter 8

How do we protect the system?

Protection Circuits

→ Input relay — inrush current rating, coil voltage, contact rating → MOV / TVS — clamping voltage, energy rating, let-through voltage → Fuse sizing — I^2t coordination, opening time vs inrush → Soft-start coordination with control loop (Chapter 6 cross-reference) → Supplier data: MOV V-I curves, energy absorption ratings

Chapter 9

How do we meet conducted emission limits?

EMI Filter Design

→ Conducted emission limit (CISPR 32 / EN 55032) — Class B limit table → Differential-mode noise source — ΔI_L and $2 \times f_{sw}$ cancellation (Chapter 3 K(D)) → Common-mode noise source — dV/dt switching edges → DM filter design — L_{DM} , C_X sizing for required attenuation at 150 kHz → CM filter design — L_{CM} , C_Y sizing, safety leakage current constraint → Full filter schematic and insertion loss verification

Part 4 — Simulation Integration Architecture

Simulation is a powerful addition but it must be integrated correctly — as a review layer, not as a replacement for the calculation engine. The wrong approach would create a parallel system that duplicates calculations and eventually diverges from the report. The right approach uses simulation to visualise and verify what the calculation engine has already computed.

✓ AGREED Simulation Sits Between Calculation and Approval — It Is a Review Step

Every simulation instance follows the same pattern as ReviewMagnetics.tsx today: (1) the calculation engine runs first and produces a result, (2) the simulation visualises that result interactively, (3) the designer validates the simulation output, (4) on approval the simulation figures become report figures. Simulation never runs before the calculation. Simulation never feeds back into the calculation.

SIMULATION The Three Simulation Types and Their Integration Points

Type A — Waveform Simulation: time-domain waveforms at a chosen operating point. Already implemented for magnetics (ReviewMagnetics JS studio). Will extend to semiconductor switching waveforms (Chapter 7) and input current waveforms with and without EMI filter (Chapter 9). Type B — Frequency-Domain Simulation: Bode plots, impedance vs frequency, EMI emission spectrum. Already implemented for control loops (FAN9672 tool). Will extend to filter insertion loss verification (Chapter 9). Type C — Thermal Simulation: temperature vs time, heatsink cooling curves. New for Chapter 7 (semiconductor junction temperature). Will use matplotlib-generated plots from Python calculation, not interactive JS.

4.1 Technical Architecture for Simulation

Sim Type	Engine	Injection Method	Report Output	Existing Example
Waveform (time domain)	JavaScript in browser (iframe, public/ folder)	postMessage with Python-computed parameters (same as ControlDesign.tsx)	Designer screenshots captured as PNG; or Python re-generates from same params	ReviewMagnetics ControlDesign
Frequency domain (Bode)	JavaScript in browser (iframe, public/ folder)	postMessage — same architecture	Python re-runs sweep with JS-extracted component values (Step 16 fix pattern)	FAN9672 tool Step 16
Thermal (static)	Python backend (matplotlib)	No iframe needed — result returned as plot image in API response	Matplotlib figure embedded in report generator directly	All current plots in Chapters 3,4
SPICE-like (future)	Python PySpice or LTSpice API	Backend runs simulation, returns waveform JSON to frontend viewer	Key waveforms extracted from simulation JSON and plotted by report generator	Not yet built

4.2 Simulation vs. Calculation — Clear Boundary

The boundary between calculation and simulation must never blur:

Calculation Engine Does	Simulation Does
Computes the inductance, crossover frequency, capacitance — the design values	Visualises the waveforms, Bode plots, step response at those design values
Runs in milliseconds on the backend — deterministic, reproducible	Runs in the browser or backend — interactive, visual
Output: numbers and tables used in design decisions	Output: figures that verify the numbers are physically reasonable
Always runs. Never skipped. Report requires it.	Optional for the designer. Can be skipped. Report uses Python-generated plots.
Defines what is approved	Confirms what is approved looks correct

CAUTION Do Not Make Report Generation Depend on Simulation

The combined report (`generate_combined_report.py`) must always be able to run from the approved result dict alone — even if the designer never opened the simulation review page. If the report generator requires simulation output, a designer who approves directly without reviewing simulation gets an incomplete report. Solution: Python always has its own plot-generation path from the calculation data. The simulation review page is a bonus visualisation, not a prerequisite.

Part 5 — Checklist for Adding Any Future Chapter

The following checklist applies to every future chapter — Chapter 7, 8, 9, or beyond. Following it in order prevents the most common integration mistakes.

Phase	Task	File(s) Created/Modified	Test
1 Define contracts	Write the TypeScript interface for ApprovedXxxResult. Write the Pydantic model in Python. Include the complete selected_xxx block with all data needed for report generation.	New TypeScript interface in relevant .tsx New Pydantic model in main.py	TypeScript compiles
2 Calculation module	Create backend/app/mode_b/step{N}_{topic}.py. Contains only maths — no PDF, no HTTP. Returns a plain dict matching the approved result schema.	step{N}_{topic}.py (new)	Unit tests pass
3 Report generator	Create generate_step{N}.py. Takes only the approved result dict as input. Returns a list of ReportLab flowables. Apply all documentation standards (splash page, equations, DATA SOURCE boxes, etc.)	generate_step{N}.py (new)	generate_step{N}_section(dict) runs without error
4 Orchestra tor	Add 3 lines to generate_combined_report.py: import, call, append. Add step{N}_result parameter with default None.	generate_combined_report.py (3 lines added)	Full report still generates for Steps 1–(N-1) with new param = None
5 API endpoint	Add POST /mode-b/step{N}/run to main.py. Update generate-full-report endpoint to accept step{N}_result.	main.py (1 new route + 1 param added)	curl test returns 200 with valid JSON
6 API client	Add step{N}RunCalc and export from client.ts.	client.ts (2 lines added)	TypeScript compiles
7 Frontend component	Create Step{N}Wizard.tsx. Define ApprovedXxxResult interface (with complete selected_xxx block). onApprove callback stores full result in App.tsx state.	Step{N}Wizard.tsx (new)	Component renders, API call works
8 App routing	Add step{N} to App.tsx Step type. Add approvedXxxResult to AppState. Add handleStep{N-1}Approve. Render Step{N}Wizard at step{N}. Update handleStep{N}Approve to pass step{N}_result to report.	App.tsx (4 additions)	Full workflow runs end-to-end
9 Stepper	Add step{N} entry to Stepper.tsx with title and icon.	Stepper.tsx (1 entry)	Breadcrumb shows all steps

10 Simulation (if applicable)	If this chapter has a review/simulation step: create HTML tool in src/assets/, copy to public/, add postMessage bridge, create Review{Topic}.tsx. Python re-generates plots from same params without needing sim output.	public/{topic}.html Review{Topic}.tsx	Simulation runs independently of report generation
11 Documentation agent	Run Documentation Agent on generate_step{N}.py. Apply all agreed standards: chapter splash, equations, DATA SOURCE boxes, structural elements, technical verbiage, graph and table standards.	generate_step{N}.py (documentation enriched)	Generated report chapter matches documentation standards PDF
12 DEVELOPMENT GUIDE.md	Update DEVELOPMENT_GUIDE.md: new step added, contract documented, any non-standard patterns explained.	DEVELOPMENT_GUIDE.md	Guide is current

Part 6 — Future Chapter Content Outlines

The following outlines define the key calculations, data sources, and report sections for each planned future chapter. These are planning guides, not final specifications.

6.1 Chapter 7 — Power Semiconductors

Key data sources required:

- **MOSFET datasheet:** V_{DSS} , $I_{D(max)}$, $R_{DS,on}(T_j)$, Q_g , C_{oss} , reverse recovery of body diode
- **Bridge rectifier datasheet:** V_{RRM} , $I_F(avg)$, I_{FSM} (surge), V_F , thermal resistance
- **Heatsink datasheet:** thermal resistance θ_{sa} (junction-to-ambient)

Calculation sections:

- 7.1 Bridge rectifier — average current, RMS current, peak repetitive current, V_R rating
- 7.2 Bridge rectifier — conduction loss ($P = I_{rms}^2 \times R_{on} + I_F \times V_F$) and junction temperature
- 7.3 MOSFET selection — V_{DS} rating ($V_{out} + V_{spike}$), $I_{D,pk} = I_{L,pk}$
- 7.4 MOSFET conduction loss — $I_{rms}^2 \times R_{DS,on}(T_j)$ — temperature-iterated
- 7.5 MOSFET switching loss — $E_{on} + E_{off}$ at f_{sw} — from datasheet energy curves
- 7.6 Gate drive requirements — Q_g , drive current, bootstrap voltage
- 7.7 Heatsink sizing — total semiconductor loss $\rightarrow \theta_{sa}$ needed $\rightarrow$ part selection
- 7.8 Junction temperature verification — $T_j = T_{amb} + P \times (\theta_{jc} + \theta_{cs} + \theta_{sa}) < T_{j,max}$

6.2 Chapter 8 — Protection Circuits

Key data sources required:

- **MOV datasheet:** $V_c(I)$ clamping curve, energy absorption W_{TM} , maximum peak current I_{peak}
- **Relay datasheet:** $I_{contact(max)}$, I_{coil} , operate time, release time, inrush current rating
- **Fuse datasheet:** I^2t melting, breaking capacity, voltage rating

Calculation sections:

- 8.1 Input relay — inrush current at turn-on (C_O uncharged), contact I^2t vs relay rating
- 8.2 NTC inrush limiter — resistance at cold start, recovery time, steady-state loss
- 8.3 MOV selection — clamping voltage $\geq 1.2 \times V_{in,pk,max}$, energy from surge waveform (IEC 61000-4-5)
- 8.4 Fuse coordination — normal operating current vs fuse rating, fault clearing I^2t
- 8.5 Soft-start coordination with control loop — relay closure timing vs SS ramp

6.3 Chapter 9 — EMI Filter Design

Key data sources required:

- **CISPR 32 / EN 55032:** Class B conducted emission limit table (150 kHz – 30 MHz)
- **Common-mode choke datasheet:** L_{CM} , saturation current, DCR, impedance vs frequency
- **X-capacitor datasheet:** capacitance, self-resonance frequency, voltage rating
- **Y-capacitor datasheet:** capacitance, maximum leakage current (safety-critical)

Calculation sections:

- 9.1 Noise source characterisation — DM: $\Delta I_L \times K(D)$ at $2 \times f_{sw}$; CM: dV/dt at switching edges
- 9.2 Required attenuation — measured or estimated noise vs Class B limit at 150 kHz
- 9.3 DM filter — L_{DM} and C_X sizing for required attenuation at 150 kHz
- 9.4 CM filter — L_{CM} and C_Y sizing, Y-cap leakage current < 0.5 mA (safety limit)
- 9.5 Filter insertion loss verification — analytical and/or simulation-based
- 9.6 Resonance check — filter resonance must not coincide with PFC switching harmonics

Summary: The architecture principles in this document ensure that adding Chapters 7, 8, and 9 — and any simulation capabilities — requires only new files, not modifications to existing working code. The 12-step checklist in Part 5 is the implementation guide for every future addition.

*Generated by PFC AI Design Agent documentation planning session · June 2026 · Output:
C:\Users\toral\OneDrive\Desktop\pfc_ai_agent_v2\specs\PFC_Future_Expansion_Plan.pdf*